

Myntra App Anomaly Detection Platform

- [Introduction](#)
- [Scope](#)
 - [First Phase](#)
 - [Targeted Features](#)
 - [Targeted Release Phase](#)
 - [Pre-Release](#)
 - [Production](#)
- [Purpose](#)
- [Requirements](#)
 - [FPS](#)
 - [Functional Requirements](#)
 - [Non Functional Requirements](#)
 - [Benchmark & POC](#)
 - [Memory](#)
 - [Functional Requirements](#)
 - [Non Functional Requirements](#)
 - [Navigational Tracking](#)
 - [Functional Requirements](#)
 - [Non Functional Requirements](#)
 - [Signal Logger](#)
 - [Functional Requirements](#)
 - [Non Functional Requirements](#)
 - [MDP as a Sink for Anomaly Detection Platform](#)
 - [Benefits](#)
 - [Limitations](#)
 - [Approaches](#)
 - [Push Only Relevant Signals](#)
 - [Push Only To Debug a Production Issue](#)
 - [Push Only to a group of users via ABTest](#)
 - [MDP Data Throughput](#)
 - [Assumption](#)
 - [Case 1 - Target 1% user](#)
 - [Log Sharing & Report Generation](#)
- [High Level Design](#)
- [Low Level Design](#)
 - [Kotlin Interface](#)
 - [Functionalities](#)
 - [Class Diagram](#)
 - [FPS](#)
 - [Functionalities](#)
 - [Class Diagram](#)
 - [Sequence Diagram](#)
 - [Memory Tracker](#)
 - [Functionalities](#)
 - [Class Diagram](#)
 - [Sequence Diagram](#)
 - [Navigation Tracker](#)
 - [Functionalities](#)

- [Signal Logger](#)
- [Functionalities](#)
- [Class Diagram](#)
 - [Highlights](#)
- [Post Release Adoption](#)

Introduction

This document throws light on design choices and lower level details of Anomaly detection platform. Please go through the [document](#) to check high level requirements and HLD.

Scope

While the HLD explains a lot of features, this document will only be limited to FPS, Navigational and Memory tracking. More features and feature specific LLD's will keep on getting added progressively as the framework matures and more features are brought into it's ambit.

First Phase

Targeted Features

- FPS tracking
- Navigational tracking
- Memory tracking

Targeted Release Phase

Pre-Release

The anomaly detection framework would be integrated with Headspin / Automation platform to collect app vitals based on pre-defined scenario.

Production

NA

Purpose

The purpose of this framework is to

- Reduce manual effort spent on native profiling with every release.
- Allow collection of app vitals for any Pod at Page level or App Level.
- Automated profiling of apps at different stages of app development / release lifecycle.

Requirements

FPS

Functional Requirements

- Collect UI FPS as Optional
- Collect JS FPS as Optional
- FPS aggregation should be channeled to a File logger.
- Average FPS, 90th Percentile, minimum and maximum FPS should be derived.
- Ability to start / stop FPS at any point in time.

- Ability to switch between multiple loggers.

Non Functional Requirements

- Minimum deviation between custom FPS collection and the standard way of collecting FPS for a particular platform.
- Minimal impact on app performance in terms of memory and cpu trends.
- The FPS recording should be bundled in a framework to be used independently if need be.
- Minimal code footprint ensure minimal impact on app size.

Benchmark & POC

Please got through the [document](#) to get details on FPS benchmark and POC details.

Memory

Functional Requirements

- Collect memory signals
- Derive resident memory, maximum memory, minimum memory and 90th percentile.
- Ability to start / stop memory collection signals at any point in time.
- Ability to log at multiple levels. File logger would be targeted in first phase.
- iOS would only have lite version of collecting memory signals.
- Android would have a lite and an extensive(similar to hprof) memory capturing mode.

Non Functional Requirements

- Minimal and predictable impact of app Vitals like cpu and memory.
- Minimal and predictable deviation from standard memory collection way on a particular platform.
- Independent lib to isolate the implementation.
- Minimal code footprint.

Navigational Tracking

Functional Requirements

- Collect page entry
- Collect page exit
- Collect navigation start
- Collect navigation end
- Each associated signal will accompany timestamp.
- HOC to handle the Anomaly features at encapsulated level

Non Functional Requirements

- JSI to reduce latency between native and JS
- Minimal impact on RN feature specific implementation
- No Perf impact on the pages on which it is integrated in.

Signal Logger

Functional Requirements

- Capture Signals from a source
- Channel it to a logger like file logger, network logger, console logger.
- Ability to log to a multiple logger / appender.
- Multithreaded capability to offload task to a different thread.
- Atomic read / write.
- Ability to log to multiple files / appenders based on multiple sources.
- Higher throughput on both read / write.
- Ability to log signals in real time or batch it.
- Ability to change batch size, log frequency.
- Handle write failures gracefully.
- Handle network failures.
- Log rotation should be made available in case of file writes.

Non Functional Requirements

- Minimal impact on app memory.
- No Impact on App performance.
- High Frequency / low latency read / writes. Should be at least better or similar to sqlite.
- Maintain atomicity of transactions
- Schema on Write would be preferred as logging would have to be fast. The report generation would be done outside the application and hence it can be slower in comparison.
- The reads should not be extremely slow as later requirements change / new feature addition can lead to reads in future.

MDP as a Sink for Anomaly Detection Platform

Based on the outputs of initial phases, a call would be taken if anomaly data should be pushed in to MDP.

Benefits

- Allows for debugging issues in production by enabling for specific set of users.
- Running experiments on specific cohort of devices(low, mid, high) to push cohort specific performance features.
- Historical data wrt experiments would help in making better decisions on performance front.

Limitations

- App has more than 10 millions users during BAU. Amount of data generated would huge if it is enabled without any limitations.
- **TCO** theme limits the usage of events on Production.

Approaches

Push Only Relevant Signals

- Scenarios should be predefined.
- The thresholds for each scenario should be defined.
- The signals should be sent to MDP only if the thresholds are breached.

Push Only To Debug a Production Issue

Restrict the usage of MDP only to debug high priority issues in production where in performance aspect of the anomaly is also required.

Push Only to a group of users via ABTest

- Run Experiments based on device, os version to identify certain behaviours and act accordingly.

MDP Data Throughput

Assumption

FPS anomaly per page - 3

Navigation Anomaly per session - 6

Memory Anomaly per page - 3

Case 1 - Target 1% user

DAU on App Platform - 11M

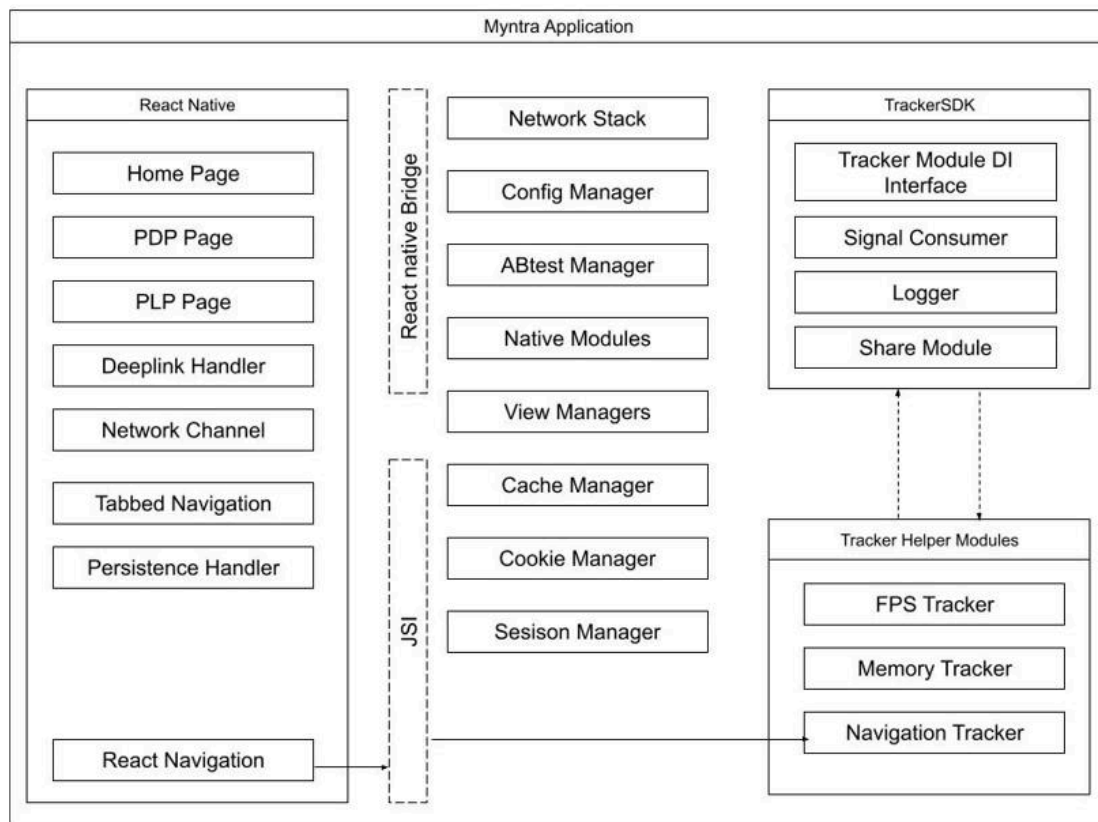
1% of DAU - 1,10,000

Log Sharing & Report Generation

Log Files would be shared via Share Module in CSV format in first phase.

High Level Design

Refer the [Document](#).



Low Level Design

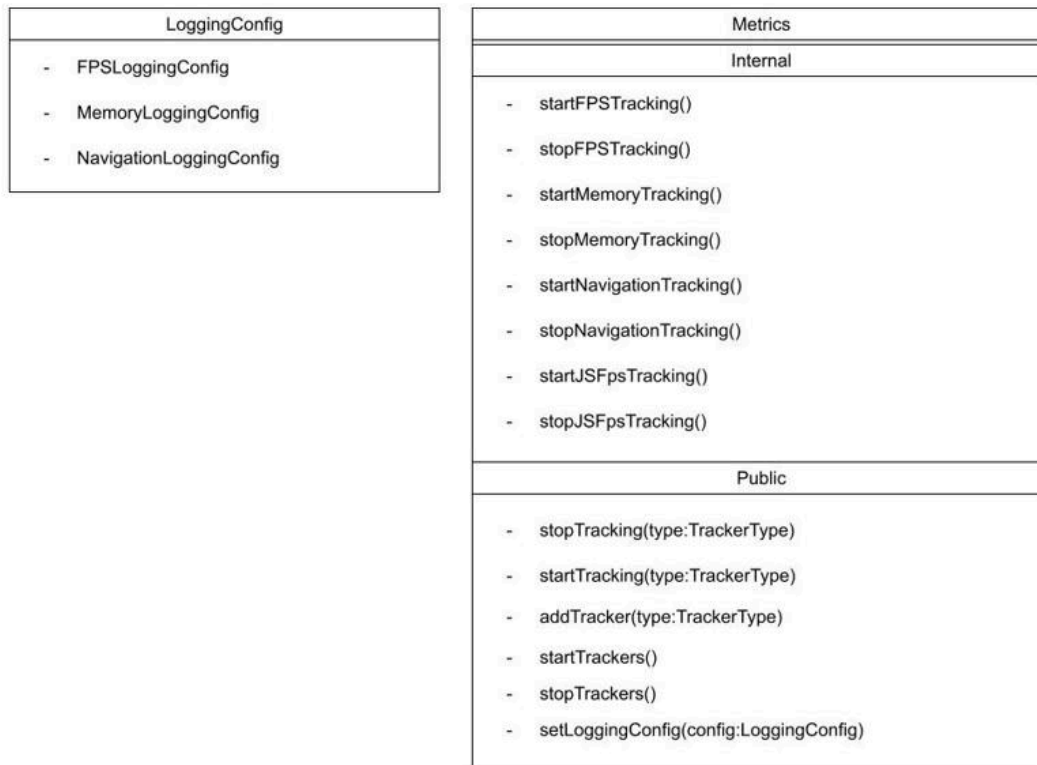
Kotlin Interface

The respective trackers like FPS, Memory and Navigation would be injected at runtime into the framework via interfaces. The said trackers would be available for perusal at runtime.

Functionalities

- Interact with FPS, Memory and Navigation Modules via interface.
- Trackers Modules would be made available via DI at runtime.
- Manage lifecycle of trackers like instantiation, starting, stopping, disposal etc.
- Observer change in tracking events of respective modules.
- Channel tracking signals to respective loggers.
- Add Navigation Tracking automatically if any of the trackers is added.

Class Diagram



FPS

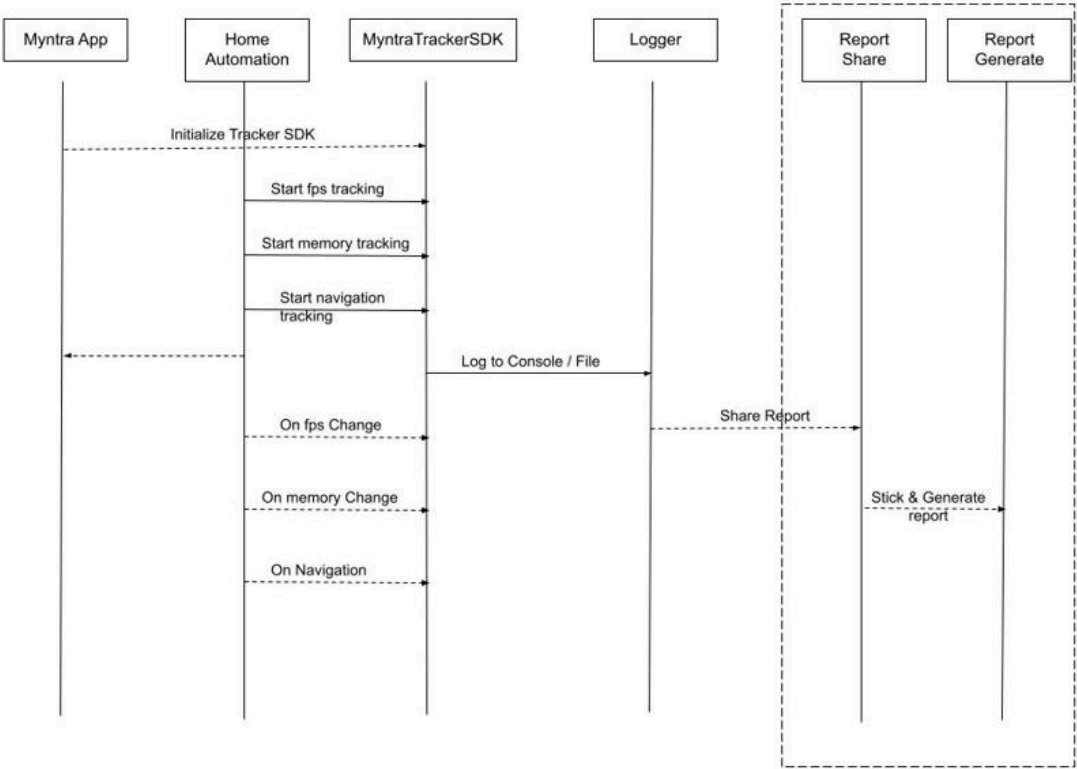
Functionalities

- Collect UI FPS
- Collect JS FPS
- Expose delegate to inform on FPS changes
- Inform Kotlin Module on FPS changes
- Log FPS changes

Class Diagram



Sequence Diagram

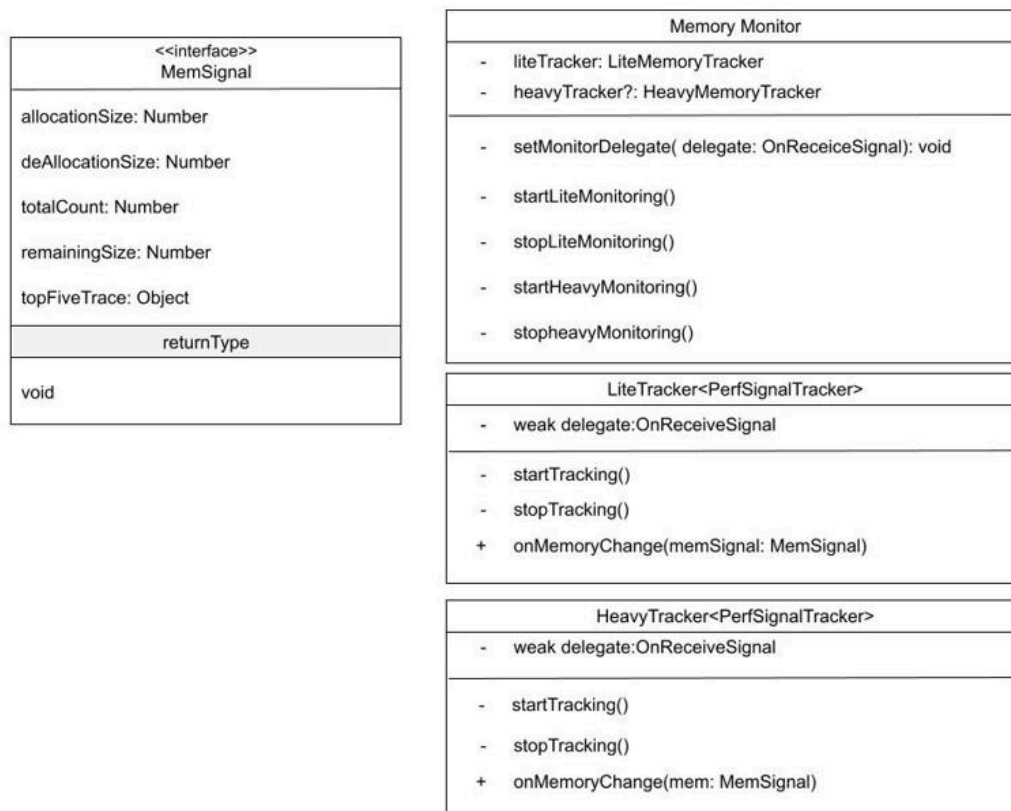


Memory Tracker

Functionalities

- Memory can be tracked in two modes Lite and Heavy.
- iOS would only have Lite version of memory tracker.
- Track Memory signals
- Inform the Tracker framework via delegate / callback.
- Log Memory Signals
- Heavy version of memory tracker would have similar granularity as **hProf**.

Class Diagram



Sequence Diagram

Similar to FPS

Navigation Tracker

Functionalities

- A React Native Module to track the page navigation
- A JSI Module to pass Navigation Signals from RN to Native.
- Native Communicator to pass signals to Tracker Framework.
- Page change can be tracker at React Navigation layer
- A new function would have to created to track Navigation Start and Stop.

Signal Logger

Most of the functionalities are already covered in the requirements category. Functional level aspects would be discussed here.

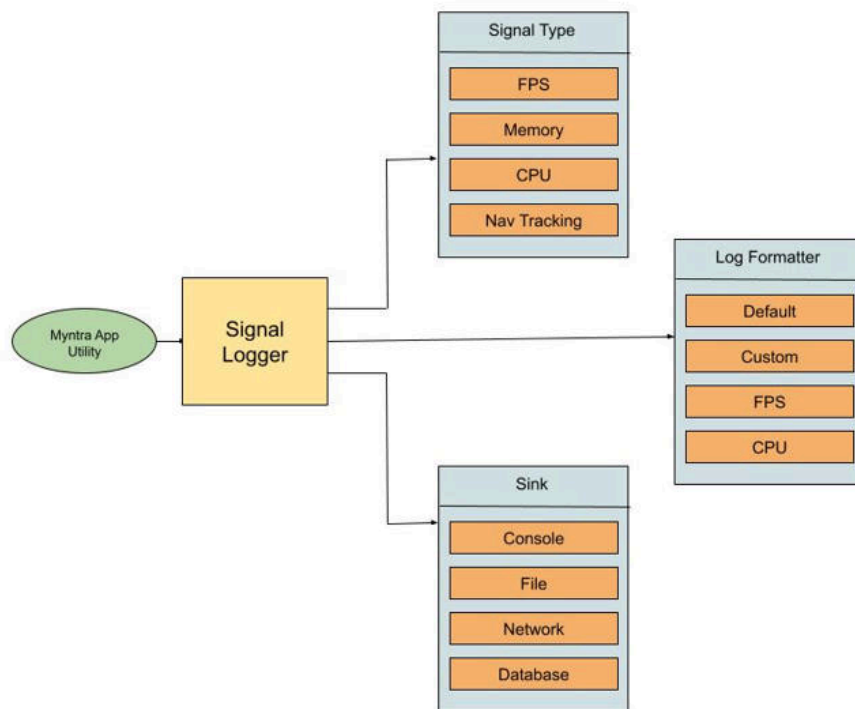
Functionalities

- Allow logging for multiple features like FPS, Nav Tracking, CPU, memory etc.
- Allow logging of multiple kinds of data.
- Allow logging to multiple destinations like console, File, Network etc.
- Log rotation should be available to handle large data logging.
- Write should be faster comparison to reads.
- NRT and batch logging should be available and configurable.
- Log can be written to multiple sinks.

Class Diagram

Highlights

- Sink would be on Factory Pattern.
- Signal Logger would be a Singleton Instance.
- Appenders would be on iterator pattern as logs can be written to multiple destinations.



<<interface>> AbstractLogFormat
+ formattedData(signal:DataType):string

<<concrete>> FPSFormat
+ formattedData(signal:number):string

<<concrete>> MemoryFormat
+ formattedData(signal:double):string

<<concrete>> NavigationFormat
+ formattedData(signal:NavigationInfo):string

<<datatype>> NavigationInfo
screenName:string time:number eventType:EventType

<<datatype>> EventType
load exit

LogFormatFactory
+ getFormattedDataForSignal(signalType:SignalType):string

<<datatype>> SignalType
UIFPS JSFPS NavTracking Memory

AbstractLogger
type: SignalType Sinks: SinkConfig
- logData(type:SignalType, msg:string): void - display(msg:string): void

FileLogger
config: FileConfig
- logData(type:SignalType, msg:string): void - display(msg:string): void

DatabaseLogger
config: DatabaseConfig
- logData(type:SignalType, msg:string): void - display(msg:string): void

ConsoleLogger
config: ConsoleConfig
- logData(type:SignalType, msg:string): void - display(msg:string): void

NetworkLogger
config: NetworkConfig
- logData(type:SignalType, msg:string): void - display(msg:string): void

<Singleton> LogManager
loggers: Loggers[]
- getInstance(): LogManager - appendSink(sinkConfig:SinkConfig) + createSink(sinkConfig:SinkConfig): Logger + log(type:SignalType, data: Any): Logger

Post Release Adoption

Please go through the [Doc.](#)